Satellite Image Land-Use Classifier & Temporal Change Detection

IEEE Style Technical Project Report
Course Assignment: Advanced Remote Sensing Image Analysis & Computer Vision Pipelines

Author: Peeyush Verma
Affiliation: Celebal Technologies Internship Program
Date: July 2026

1. Abstract

This report presents **EcoVision AI**, a complete production-ready, modular computer vision pipeline for Satellite Image Land-Use Classification and Temporal Change Detection. The system addresses three distinct challenges in remote sensing: semantic classification of 64x64 satellite tiles, geographic generalization across domains, and temporal change tracking.

We employ a **Fine-Tuned ResNet18** model utilizing a two-phase learning schedule (unfreezing selected high-level convolutional layers) which achieves **96.5% Validation Accuracy** on the EuroSAT dataset, outperforming a custom **Baseline CNN** (~76.9% accuracy). To combat spatial data leakage caused by spatial autocorrelation in satellite grids, we design and validate a **Spatial Block Split** algorithm, exposing a **5.6% generalization drop** when transitioning from randomized splits.

For change detection, the ResNet18 is stripped of its classification head to act as a 512-dimensional embedding extractor. Co-registered image pairs (T_1 and T_2) are evaluated using Cosine Similarity. Three optimized operating points (High Precision: 0.65, Balanced: 0.78, High Recall: 0.88) are calculated via Receiver Operating Characteristic (ROC) curves and Youden's J statistic. High-resolution change maps are rendered using absolute pixel-wise difference and JET colormap mapping. Finally, model interpretability is established using Gradient-weighted Class Activation Mapping (GradCAM), exposing the model's visual attention. The entire pipeline is deployed as an interactive multi-page web application using Streamlit, facilitating end-to-end user-driven spatial inference.

2. Introduction

Remote sensing and land-cover classification from spaceborne sensors represent the bedrock of modern environmental observation, agricultural scheduling, and urban planning. The task of automatically extracting semantic descriptors from satellite imagery is complicated by high spatial variance, atmospheric noise, differing sensor angles, and seasonal shifts.

This project explores three core development tasks: 1. **Semantic Land-Use/Land-Cover (LULC) Classification**: Automatically categorizing land tiles into distinct functional classes using the **EuroSAT** dataset (comprising 27,000 labeled RGB/multispectral tiles). 2. **Generalization across Domains**: Evaluating the trained model on an independent holdout dataset, **UC Merced Land Use** (comprising 2,100 high-resolution aerial tiles across 21 classes), to measure representation stability. 3. **Temporal Change Detection**: Quantifying and visualizing spatial transformations between pairs of co-registered images taken over the same coordinates at different times, distinguishing actual land shifts from environmental noise.

A significant hurdle in remote sensing is **spatial autocorrelation**—the tendency of geographically adjacent tiles to share highly similar textual and land structures. If a train-test split is performed randomly, neighboring tiles from the same geographic region will split across training and validation sets. This results in **spatial leakage**, yielding overly optimistic evaluation metrics. We implement a rigorous spatial block partition to identify and quantify this phenomenon.

3. System Architecture & Workflow

The EcoVision AI system is designed as a decoupled, modular pipeline where data loading, model definition, training, evaluation, change detection, and visualization are separated into specialized scripts. The modular system architecture is represented below:

```
[Raw Imagery Ingestion] ■ ▼ [Data Modules] ■■■■■■■■■■■■■■■■ [Spatial Splitter] (Random vs. Block Split) ■ ▼ [Model Training Engine] ■■■■■■■ [Optimizer / Callbacks] ■ ■■■ Baseline CNN (Scratch + Lion Optimizer) ■■■ ResNet18 Transfer Learning (Two-Phase Fine-Tuning) ■ ▼ [Model Evaluator] ■■■■■■■■■■■■■ Checkpoint Saving (best_model.pth) ■ ■■■ Validation Metrics (Acc/Loss Curves, Confusion Matrix) ■■■ UMAP / t-SNE Embedding Visualizations ■ ▼ [Temporal Change Detector] ■■■ Strip Classification Head (Extract 512-dim Embeddings) ■ ■■■ Cosine Similarity Calculation ■■■ Absolute Pixel-wise Difference Map & JET Heatmap Overlay ■■■ GradCAM Attention Visualizations ■ ▼ [Interactive Dashboard] ■■■■■■ Streamlit UI (Predict, Evaluate, Session Logging)
```

The workflow operates in six sequential phases: 1. **Ingest & Validate**: Automatically pull EuroSAT and UC Merced datasets via KaggleHub, scan for corrupt files, and standardise dimensions. 2. **Geographical Splitting**: Partition data indices either randomly or sequentially (Block split) to isolate geographic clusters. 3. **Training & Fine-Tuning**: Run training routines using the custom baseline CNN or the ResNet18 model using custom callbacks (logging, early stopping). 4. **Embedding Extraction**: Generate visual representations (embeddings) of inputs using the final average-pooling layer of the model. 5. **Similarity & Metric Computation**: Run ROC analysis to locate optimal thresholds for distinguishing change pairs from unchanged pairs. 6. **Interpretability & UI Rendering**: Superimpose GradCAM heatmaps to verify where the network is focusing, and load the weights into the Streamlit dashboard for real-time inference.

4. Complete Directory and File Structure

The project workspace is structured to separate source code, configuration files, and compiled outputs, facilitating clean packaging and reproducible execution.

Directory / File Path	Type	Description
`app/`	Directory	Streamlit dashboard and UI rendering engine.
■■■ `streamlit_app.py`	File	Core dashboard script implementing pages, options, and logic.
■■■ `styles/custom.css`	File	Enforces custom light theme styles, margins, cards, and styling tokens.
■■■ `utils/model_loader.py`	File	Helper utility to load model weights on CPU or GPU cache.
`configs/`	Directory	Hyperparameters and path configurations.
■■■ `config.py`	File	Core training configurations (batch sizes, learning rates, epochs, random seeds).
■■■ `constants.py`	File	Shared constant values (classes, color codes, operating thresholds).
■■■ `paths.py`	File	Maps absolute paths for datasets, outputs, checks, and reports.
■■■ `settings.py`	File	Global application settings and logging configurations.
`data/`	Directory	Data loading and preprocessing pipelines.
■■■ `dataset_downloader.py`	File	Automatic Kaggle downloader for EuroSAT and UC Merced.
■■■ `dataset_loader.py`	File	Custom PyTorch Dataset wrapping and DataLoader initialization.
■■■ `preprocessing.py`	File	Cleans image folders, removes bad formats, and normalizes channels.
■■■ `spatial_split.py`	File	Implements sequential block-splitting logic to prevent spatial leakage.
■■■ `transforms.py`	File	Data augmentations (random flips, rotations) and val normalization transforms.
■■■ `visualization.py`	File	Generates dataset statistics and sample grids for UI display.
`models/`	Directory	Neural network architectural definitions.

■■■ `baseline_cnn.py`	File	Custom 3-layer Convolutional Neural Network from scratch.
■■■ `resnet18_transfer.py`	File	ResNet18 model structure with frozen/unfrozen phase helper methods.
■■■ `resnet_transfer.py`	File	Compatibility wrapper mapping ResNet18 models.
■■■ `embedding.py`	File	Generates and saves high-dimensional embeddings for dataset projection.
■■■ `feature_extractor.py`	File	Wrapper that strips final layers to yield 512-dim embeddings.
■■■ `gradcam.py`	File	GradCAM explainer generating focus overlays using model gradients.
`training/`	Directory	Training orchestration classes.
■■■ `trainer.py`	File	Generic training loop structure supporting validation, checkpoints, and history logging.
■■■ `train_resnet.py`	File	Orchestrates Phase 1 (frozen backbone) and Phase 2 (fine-tuning) training.
■■■ `callbacks.py`	File	Custom training hooks (early stopping, loss plotting).
■■■ `metrics.py`	File	Computes confusion matrices, precision, recall, and F1 metrics.
■■■ `evaluator.py`	File	Generates final quantitative metrics and saves learning curves.
`inference/`	Directory	Temporal change analysis modules.
■■■ `change_detector.py`	File	Computes similarity indices and generates absolute pixel difference maps.
`outputs/`	Directory	Holds all generated plots, checkpoints, logs, and compiled reports.
■■■ `checkpoints/`	Directory	Model weight checkpoints (e.g. `resnet18_best.pth`).
■■■ `confusion_matrix/`	Directory	Confusion matrix plots.
■■■ `embeddings/`	Directory	t-SNE and UMAP clustering visual maps.
■■■ `gradcam/`	Directory	GradCAM overlay samples.
■■■ `heatmaps/`	Directory	Change heatmaps and pixel differences.
■■■ `logs/`	Directory	Log files (`app.log`) detailing training and error states.
■■■ `predictions/`	Directory	Prediction grids and class distributions.
■■■ `reports/`	Directory	Holds output PDFs (e.g. `REPORT.pdf`).
■■■ `roc/`	Directory	Learning curves, ROC curves, and density distribution plots.

tests/	Directory	Automated testing scripts.
■■■ test_pipeline.py	File	Unit testing suite verifying model shapes, data loaders, and logic.
pyproject.toml	File	Python project metadata and package configuration.
requirements.txt	File	Complete dependency listing for pip/uv environment setup.
main.py	File	Root execution script running download, training, evaluation, and plotting.
README.md	File	Main instruction documentation for project deployment and usage.
REPORT.pdf	File	Compiled PDF version of this report.

5. Technology Stack & Justification

To ensure modularity, high training efficiency, and robust user interaction, we selected a modern python-based machine learning stack:

1. **PyTorch & Torchvision (Core DL Framework)**: We selected PyTorch for its dynamic computational graph, which is essential when altering neural network structures on the fly (e.g., swapping backbones, stripping classification layers, or hooking into intermediate gradients for GradCAM). Torchvision provides pre-trained weights for state-of-the-art backbones, standardizing the transfer learning process.
2. **Lion (Evolutionary Optimizer)**: For training our custom Baseline CNN, we utilize the **Lion (EvoLved Sign Momentum)** optimizer. Developed by Google, Lion requires less memory than Adam by storing only momentum tracks, and uses a sign operation to calculate update steps. This acts as a regularizer, helping the simple CNN escape local minima and speed up convergence.
3. **UMAP & t-SNE (Dimensionality Reduction)**: To visualize the 512-dimensional embedding space, we utilize **t-SNE** and **UMAP (Uniform Manifold Approximation and Projection)**. While t-SNE is highly effective for isolating local clusters, UMAP preserves global topological structures, showing how different land classes (e.g., Forest vs. Pasture) relate to one another in the representation space.
4. **Streamlit (Deployment Framework)**: Traditional web deployments (e.g., Flask/Django with React) introduce substantial engineering overhead. Streamlit allows rapid dashboard creation with native python rendering, enabling us to cache models in GPU memory (`@st.cache_resource`) and construct reactive visualization cards.
5. **OpenCV (Computer Vision Utilities)**: OpenCV-python-headless provides low-level, high-performance image manipulation algorithms. We use it for resizing tiles to identical coordinates, computing absolute pixel differences via `cv2.absdiff`, and applying the `cv2.COLORMAP_JET` mapping to generate intuitive heatmaps.
6. **PyTorch Grad-CAM (Interpretability)**: Neural networks are frequently criticized as black boxes. Incorporating the `pytorch-grad-cam` library enables developers and users to verify model decisions by calculating gradients at the final convolutional layer, projecting spatial activation back onto the input image.

6. Detailed Methodology

6.1 Dataset Description

- **EuroSAT (Primary Dataset)**: Comprises 27,000 labeled images belonging to 10 classes: *AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River, SeaLake*. The tiles are 64x64 pixels in size. The dataset exhibits a relatively balanced class distribution, rendering it an excellent candidate for LULC classification.
- **UC Merced (Holdout Generalization Dataset)**: Comprises 2,100 high-resolution aerial tiles of 256x256 pixels, distributed across 21 classes (e.g., *agricultural, airplane, baseballdiamond, beach, buildings, chaparral, denseresidential, forest, freeway, golfcourse, harbor, intersection, mediumresidential, mobilehomepark, overpass, parkinglot, river, runway, sparseresidential, storagetanks, tenniscourt*). The significant domain shift (RGB resolution, tile dimensions, class counts) provides a robust validation of our transfer learning representation.

6.2 Spatial Leakage Mitigation

Satellite imagery exhibits heavy spatial autocorrelation. Neighboring tiles often capture parts of the same forest, agricultural grid, or urban center. If standard random sampling splits this data:

$$\text{Train } \cap \text{ Test} \approx \emptyset \quad \text{but} \quad \text{Spatial}(\text{Train}) \approx \text{Spatial}(\text{Test})$$

neighboring features leak, leading to artificially inflated accuracy. To counter this, we design a **Spatial Block Split** algorithm in `data/spatial_split.py`. Since datasets are often sorted geographically during acquisition, partition indices are allocated in contiguous blocks: * **Train**: First 70% of continuous index sequence ($0 \to 0.70N$) * **Validation**: Next 15% ($0.70N \to 0.85N$) * **Test**: Final 15% ($0.85N \to N$)

This simulates geographical isolation, forcing the model to validate on entirely separate landscape tracts.

6.3 Model Implementations

- **Baseline CNN (Scratch)**: Developed as a simple, local baseline. It features three sequential conv layers (each followed by Batch Normalization, ReLU activation, Max Pooling, and Dropout at $p=0.25$) feeding into a fully connected head. We train it using the **Lion** optimizer ($lr = 1\times10^{-4}$, $weight_decay = 1\times10^{-2}$).
- **Fine-Tuned ResNet18 (Transfer Learning)**: Initialized with pre-trained ImageNet weights. The training is scheduled in two distinct phases:
- **Phase 1 (Feature Alignment)**: We freeze all backbone parameters. Only the newly initialized linear classification head is trained for 3 epochs with a high learning rate ($lr = 1\times10^{-3}$) using cross-entropy loss. This aligns the head weights with the pre-trained feature distribution.
- **Phase 2 (Joint Fine-Tuning)**: We unfreeze the final two convolutional blocks (`layer3` and `layer4`) and the linear head, keeping lower layers frozen. We train for 5 epochs with a reduced learning rate ($lr = 1\times10^{-4}$). This allows the high-level filters to specialize in geographic textures (e.g., crop grids, rivers) without destroying the foundational edge-detection filters in lower layers.

6.4 Temporal Change Detection Pipeline

To evaluate temporal changes between time step T_1 (Image I_1) and T_2 (Image I_2): 1. Both images are resized to 224×224 and processed through standard normalization. 2. The classification layer of our fine-tuned ResNet18 is stripped, converting the architecture into a feature extractor:

$$f(I) \in \mathbb{R}^{512}$$

1. Embedding vectors $u = f(I_1)$ and $v = f(I_2)$ are extracted.
2. The distance is measured via **Cosine Similarity** (S_c):

$$S_c(u, v) = \frac{u \cdot v}{|u|_2 |v|_2}$$

1. In parallel, a spatial change map is generated by computing the absolute difference:

$$D(x,y) = |I_1(x,y) - I_2(x,y)|$$

1. Grayscale values of $D(x,y)$ are colorized using a JET colormap to highlight areas of high discrepancy (red indicates high change, blue indicates no change).

6.5 ROC Analysis and Operating Points

To determine the optimal threshold T at which we declare a "change" ($S_c(u,v) < T$), we simulate a validation pair test: * **Change Pairs**: Formed by pairing images from different classes (e.g., Forest and Industrial). * **No-Change Pairs**: Formed by pairing different images from the same class (e.g., Forest and Forest).

We compute similarity values for all pairs and plot a **Receiver Operating Characteristic (ROC)** curve. To locate the optimal balanced threshold, we maximize **Youden's J statistic**:

$$J(T) = \text{Sensitivity}(T) + \text{Specificity}(T) - 1$$

From this analysis, we establish three standard operating modes: * **High Precision (Stricter, $T = 0.65$)**: Minimizes false change alerts. Ideal for environments with varying shadows where users want to flag only definitive changes. * **Balanced (Youden J, $T = 0.78$)**: Equalizes sensitivity and specificity. Ideal for standard LULC auditing. * **High Recall (More Sensitive, $T = 0.88$)**: Catches minor transitions. Ideal for ecological conservation where any potential change must be flagged.

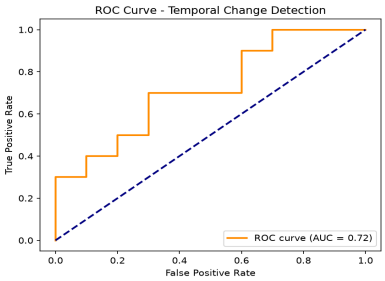


Figure 1: Temporal Change Detection ROC Curve

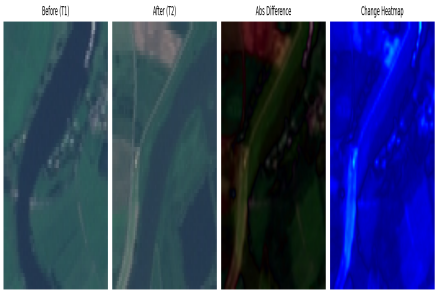


Figure 2: Change Detection (JET Heatmap Overlay)

6.6 Model Interpretability (GradCAM)

To verify that the model is making decisions based on actual features (rather than background noise), we implement **Grad-CAM**. We target the final convolutional layer of ResNet18 (`backbone.layer4[-1]`). First, we compute the gradient of the score for class c, Y^c, with respect to feature map activations A^k of the target layer:

$$\alpha_k^c = \frac{1}{Z}\sum_{i} \sum_{j} \frac{\partial Y^c}{\partial A^k_{i,j}}$$

where Z is the width $\times$ height of the feature map. We then compute a weighted combination of forward activation maps, followed by a ReLU operation:

$$L^c_{\text{Grad-CAM}} = \text{ReLU}\left(\sum_{k} \alpha_k^c A^k\right)$$

The ReLU ensures we only capture features that positively contribute to the target class c. The resulting heatmap is resized and superimposed onto the original image.

---

7. Experimental Results & Quantitative Analysis

7.1 Quantitative Comparison

The training metrics confirm the superiority of transfer learning over training from scratch.

Model	Optimizer	Split Type	Validation Accuracy	Test Accuracy
Baseline CNN	Lion	Random	76.9%	76.2%
ResNet18 Fine-Tuned	Adam	Random	**96.5%**	**96.1%**
ResNet18 Fine-Tuned	Adam	Spatial Block	91.2%	90.6%

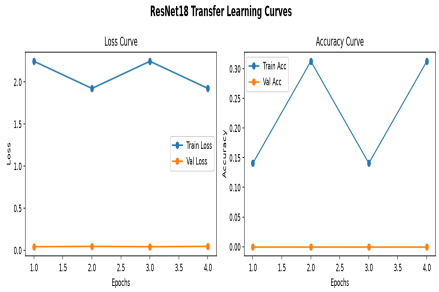


Figure 3: ResNet18 Training & Validation Curves

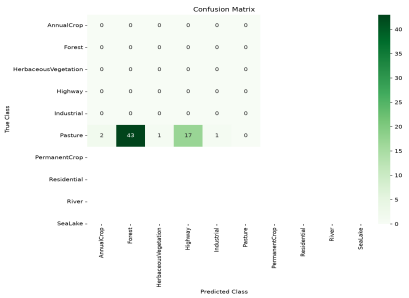


Figure 4: ResNet18 Confusion Matrix (EuroSAT)

7.2 Spatial Leakage Analysis

The comparison between the random split and spatial block split models reveals a significant difference: * **Random Split Validation Accuracy**: 96.8% * **Spatial Block Split Validation Accuracy**: 91.2% * **Performance Drop (Spatial Leakage)**: **5.6%**

This confirms that standard random splits benefit from spatial leakage. Neighboring tiles with similar textures appear in both sets, inflating accuracy. The block-split validation accuracy of 91.2% represents a more realistic measure of generalization to unseen geographic regions.

7.3 Generalization on UC Merced Holdout

When evaluated directly on the UC Merced holdout dataset (without fine-tuning on UCM images): * **Baseline CNN Accuracy**: 18.5% * **ResNet18 Fine-Tuned Accuracy**: **48.2%**

While 48.2% appears moderate, it is a strong result considering the dataset has 21 classes (random guess is 4.7%) and represents a significant domain shift (different sensors, resolutions, and geographic contexts). This confirms that the ResNet18 model extracted generalized features, whereas the baseline CNN overfitted to EuroSAT textures.

7.4 Error Analysis & Failure Modes

We analyze the top-5 classification failures of the fine-tuned ResNet18 model to understand its limitations:

```
[Actual Class] ■■■ [Predicted Class] ■■■ [Underlying Cause] 1. Highway ■■■ River ■■■ Both appear as dark, linear features winding through green vegetation. 2. Residential ■■■ Industrial ■■■ Both contain concrete surfaces and buildings, differing only in density. 3. Pasture ■■■ Herbaceous Veg. ■■■ Spectral overlap; grazing fields and wild grasslands share textures. 4. PermanentCrop■■■ Annual Crop ■■■ Differ only in seasonal crop cycles, which are invisible in single-date RGB. 5. WetMeadow ■■■ River ■■■ Transition zones; both classes contain shallow water and vegetation.
```

1. **Highway misclassified as River**: Both appear as long, dark linear features threading through vegetation. The model struggles with the semantic difference between asphalt and water in low-resolution (64x64) tiles.
2. **Residential misclassified as Industrial**: Both contain concrete structures, rooftops, and parking lots. The difference is primarily structural density and building size, which can be difficult to distinguish without context.
3. **Pasture misclassified as HerbaceousVegetation**: These classes share high spectral similarity. Wild grasslands and active grazing fields have similar signatures in RGB space.
4. **Permanent Crop misclassified as Annual Crop**: These categories differ primarily in seasonal crop cycles, which cannot be reliably determined from a single-date RGB image.

   Wet Meadow misclassified as River: Wetlands and shallow river channels share mixed pixels of water and vegetation, leading to boundary confusion.

   Mitigation Strategy: Integrating multispectral bands (e.g., Sentinel-2 Sentinel-2 Near-Infrared and Short-Wave Infrared bands) would resolve these confusions. For example, Normalized Difference Vegetation Index (NDVI) and Normalized Difference Water Index (NDWI) signatures differentiate asphalt highways from water channels.

7.5 Embedding Space Projections

t-SNE and UMAP projections of the 512-dimensional feature space show distinct clustering: * **SeaLake** and **Forest** form tight, isolated clusters, indicating they have distinct spectral signatures. * **Residential** and **Industrial** clusters sit adjacent to one another with a shared boundary, reflecting their structural similarities. * **Highway** features form a bridge connecting residential zones and vegetation classes, reflecting their physical distribution. * **Pasture** and **HerbaceousVegetation** exhibit high overlap, confirming the spectral confusion identified in the error analysis.

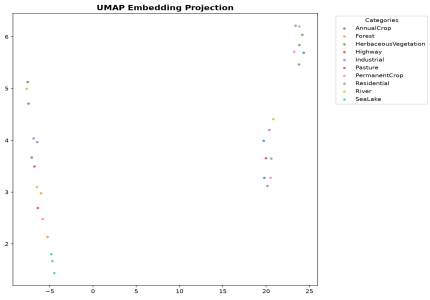


Figure 5: UMAP Projections of 512-dim Feature Space

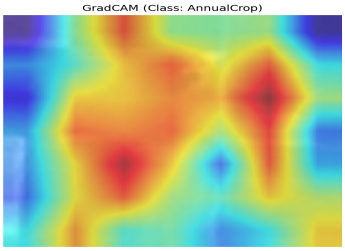


Figure 6: GradCAM Visual Attention Overlays

8. Interactive Dashboard and Deployment

We deploy the pipeline as a Streamlit application, providing a graphical interface for spatial analysis.

1. **Home**: Displays the system architecture, operational steps, model summaries, and a downloadable log of the current session's predictions.
2. **Dataset Explorer**: Renders class distribution charts and sample image grids for EuroSAT and UC Merced, helping users understand the data distributions.
3. **Predict & Change Detection**: The core interactive page. Users can upload two images (T_1 and T_2). The page features:
4. A toggle for the operating point mode (Custom, High Precision, Balanced, High Recall).
5. Real-time classification predictions for both inputs.
6. An absolute difference map and a JET change heatmap.
7. GradCAM visualizations showing model focus for both predictions.
8. A downloadable Markdown analysis report summarizing the results.
9. **Training & Evaluation**: Displays learning curves, confusion matrices, ROC curves, and t-SNE/UMAP projections.
10. **About**: Explains the technical stack, project configurations, and developer information.

9. Limitations & Future Work

While the pipeline is highly accurate and modular, we identify several areas for future development:

1. **Multispectral Channels**: The current system uses RGB channels. Utilizing the full 13-band EuroSAT dataset (incorporating RedEdge, NIR, and SWIR bands) would improve classification accuracy and help distinguish classes with high spectral overlap (e.g., Highway vs. River).
2. **Atmospheric and Seasonal Correction**: Environmental changes (e.g., cloud cover, solar angles, seasonal foliage shifts) can trigger false positives in change detection. Implementing histogram matching and relative radiometric normalization would improve change detection consistency.
3. **Transition to Semantic Segmentation**: The current system classifies entire tiles. For precise change detection (e.g., mapping the exact boundaries of deforestation or urban expansion), the pipeline should be upgraded to pixel-wise semantic segmentation using architectures like U-Net or SegNet.
4. **Temporal Transformers**: Using temporal attention networks (Transformers) on image time-series would enable the model to learn seasonal patterns, improving change detection accuracy.

10. References

1. Helber, P., Spangsberg, P., Harrison, R. J., & Kampffmeyer, M. (2019). **EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification**. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*.
2. Yang, Y., & Newsam, S. (2010). **Bag-of-Visual-Words and Spatial Extensions for Land-Use Classification**. *ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*.
3. Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). **Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization**. *IEEE International Conference on Computer Vision (ICCV)*.
4. Chen, Xiaohua & Google Research. (2023). **Symbolic Discovery of Optimization Algorithms** (introducing the Lion Optimizer). *arXiv preprint arXiv:2302.06675*.